



Clase 8

Aseguramiento de Calidad

Andres Caicedo · AI for Developers | 30X · 2h · 120 min

¿Dónde estamos en el journey?

Apertura y Contexto

- ▶ **Clases 4-5** → Inception + Construction con AI-DLC (specs, ADRs, infra design)
- ▶ **Clase 6** → Scaffolding e implementación con arneses (skills, `PRODUCT.md`, `DESIGN.md`)
- ▶ **Clase 7** → Orquestación AI-DLC → Linear, code review automatizado y memoria de proyecto
- ▶ **Hoy** → El filtro de calidad antes de producción: pruebas funcionales

 El producto existe y está orquestado. La pregunta ahora es: ¿cómo garantizas que funciona como el usuario espera?

Clase 5.5: del vibe coding al agentic engineering

Lo no funcional entra al spec como criterio con evidencia, no como ajuste al final

- ▶ **Vibe coding:** el agente produce rápido y nadie definió qué es "correcto"
- ▶ **Agentic engineering:** calidad, seguridad, observabilidad e infra se deciden antes de codificar
- ▶ **Calidad hoy · Infraestructura** Clase 9 · **Seguridad** Clase 10 · **Observabilidad** Clase 11
- ▶ **Integridad del agente:** quien produce no verifica — el agente no cambia sus reglas
- ▶ **La IA amplifica:** si produce más rápido, hay que verificar a la misma velocidad

 El vibe coding entrega demos. El agentic engineering entrega productos que resisten producción.

La tesis de hoy: de texto para humanos a interfaz de agentes

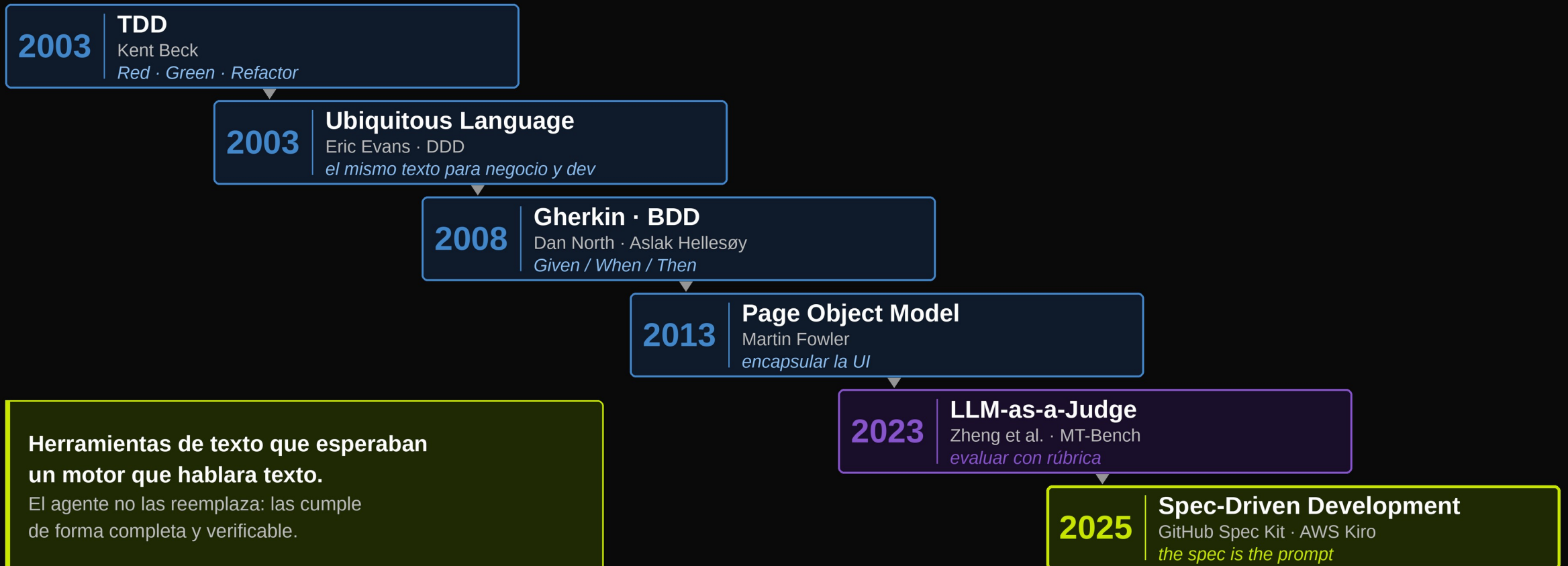
El ángulo va más allá de la velocidad

- ▶ Un bug en producción cuesta **10x más** que uno detectado a tiempo — esa es la razón obvia
- ▶ El foco de hoy: **Gherkin, POM, TDD y las rúbricas nacieron pre-IA** para que humanos se coordinaran con **texto estructurado**
- ▶ Ese mismo texto es justo lo que un agente entiende mejor que nada
- ▶ Con el artefacto correcto, el agente hace el trabajo **completo, sin drift y verificable** — no a medias
- ▶ No son legado a superar: son la **interfaz que faltaba**

💡 La IA no reemplaza estos patrones — los cumple. El texto que coordinaba humanos ahora coordina agentes.

El arco: 2003 → 2025

Herramientas de texto que esperaban un motor que hablara texto



02 — SECCIÓN

La Pirámide de Pruebas

¿Dónde invertir el esfuerzo de calidad?

La Pirámide de Pruebas

Del mayor volumen al mayor valor



Dónde invertir el esfuerzo

Trade-offs y el anti-patrón del Ice Cream Cone

- ▶ **Base amplia de unitarios:** rápidos, baratos, dan feedback inmediato
- ▶ **Capa media de integración:** validan que los componentes se comunican
- ▶ **Punta de E2E:** pocos pero de alto valor — los que el usuario siente
- ▶ **Anti-patrón "Ice Cream Cone":** invertir la pirámide — todo E2E, pocos unitarios → frágil y lento

 En este curso: unitarios e integración se cubren con tu pipeline de Construction (clases 4-7) · E2E + API + Agentes → hoy

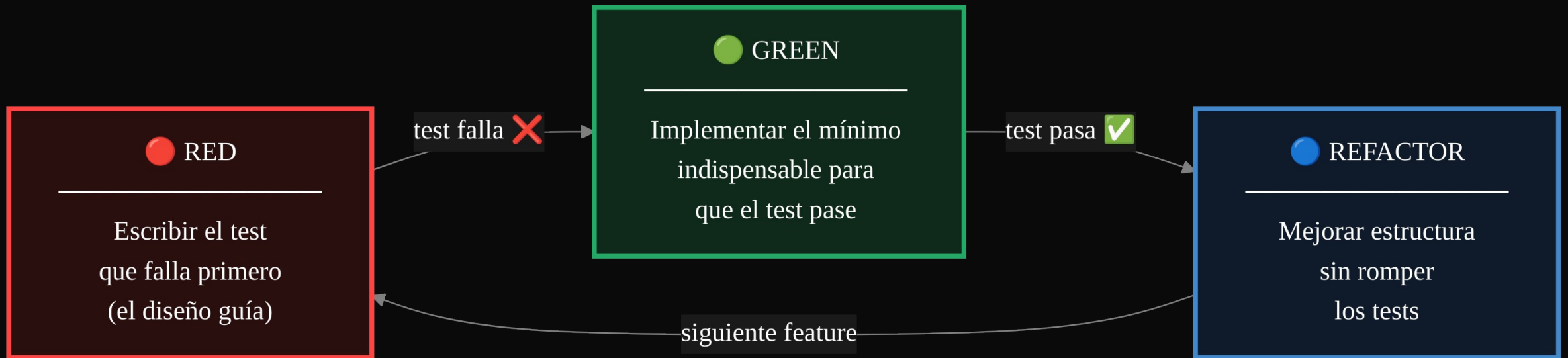
03 — SECCIÓN

BDD, TDD y Gherkin

Comportamiento como especificación ejecutable

TDD: Red → Green → Refactor

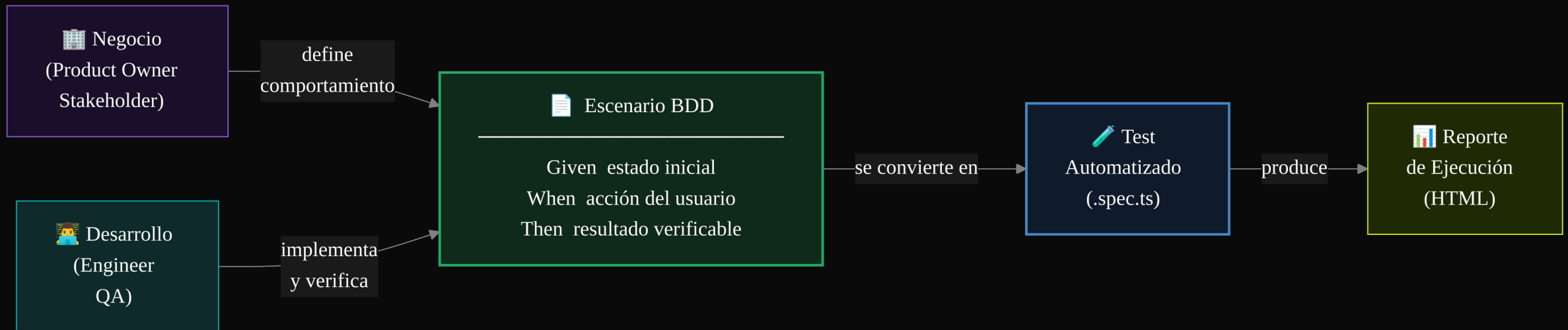
Kent Beck (2003) — el ciclo que guía el diseño, no solo la verificación



💡 TDD no es sobre tests — es sobre diseño. El test es la especificación de lo que el código debe hacer.

BDD: Comportamiento como especificación

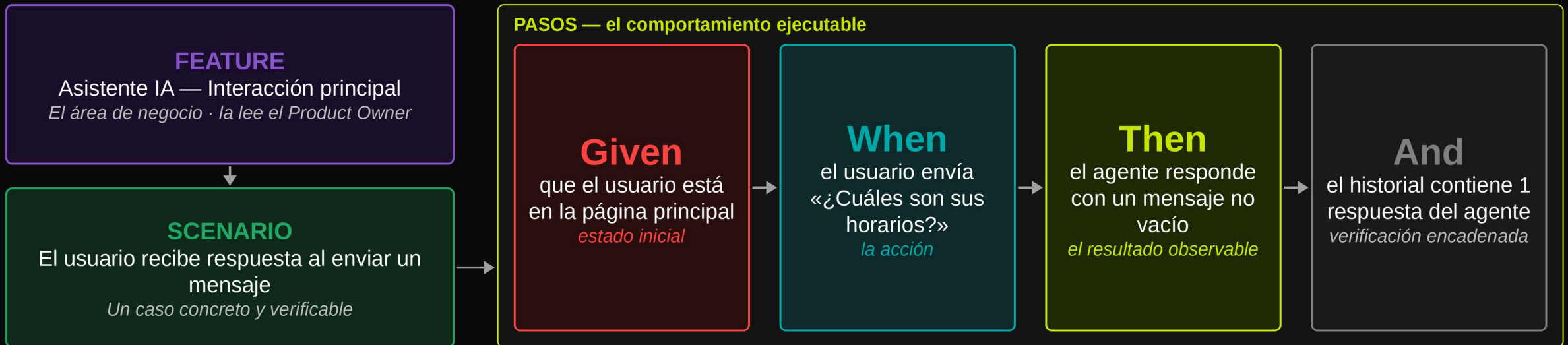
Dan North (2003) + ubiquitous language de Eric Evans — negocio y dev hablan igual



💡 BDD aplica a flujos web, contratos de API y conversaciones de agentes. El negocio puede leer y validar los escenarios.

Gherkin: El .feature file

Aslak Hellesøy · Cucumber (2008) — anatomía de un escenario ejecutable



💡 Un `.feature` es *Spec-Driven Development* 17 años antes de tener nombre. "The spec is the prompt" (2025) — Gherkin ya lo hacía en 2008.

DEMO

Darle al agente **solo** `tests/features/asistente.feature` → generar los step definitions completos → ejecutar `npx playwright test tests/bdd/` → revisar reporte HTML. La spec fue el prompt.

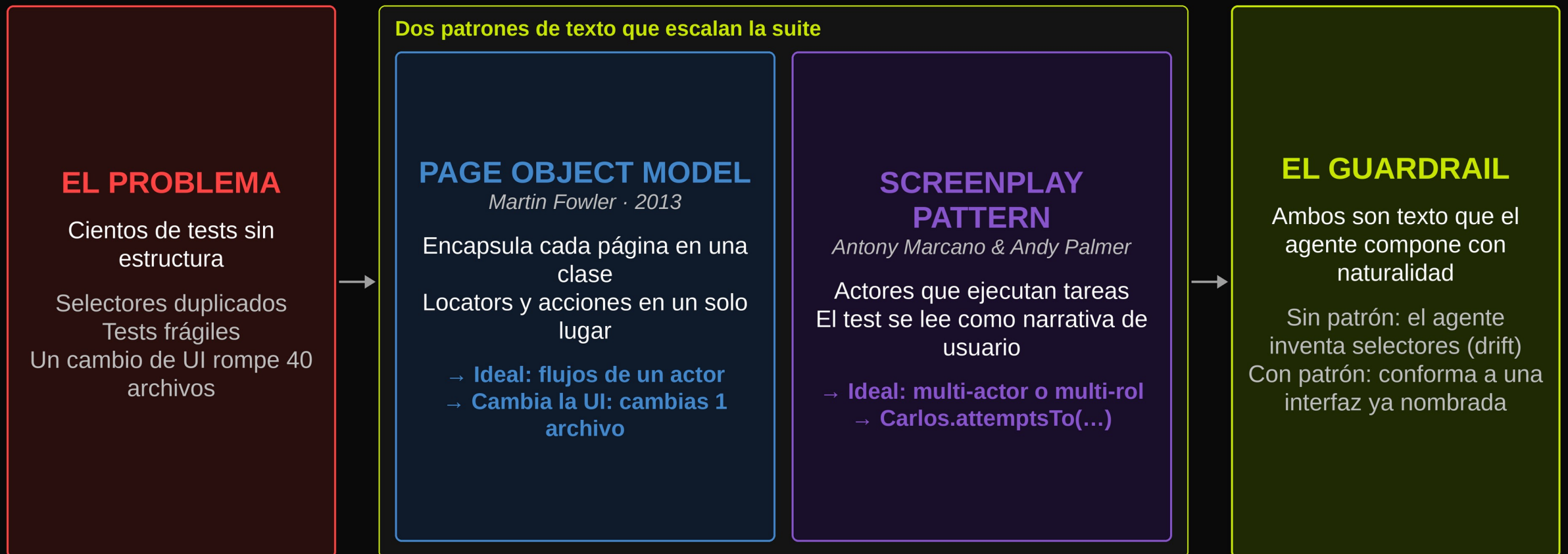
04 — SECCIÓN

Patrones de Diseño para Testing

Cómo escalar sin que el mantenimiento te consuma

Dos patrones para tests que escalan

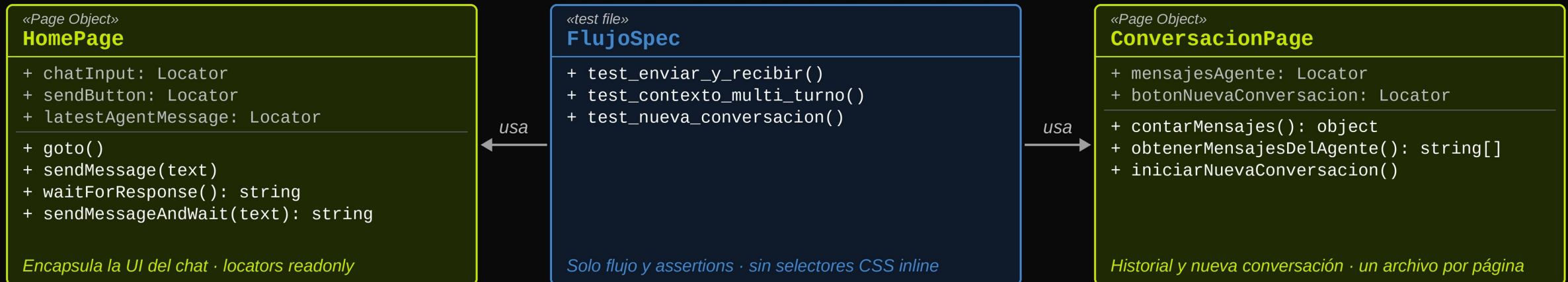
POM y Screenplay — el patrón es el guardrail del agente



Page Object Model (POM)

Martin Fowler (2013) — encapsular la UI en clases reutilizables

Qué es: un patrón de diseño que representa cada página de la aplicación como una clase. La clase guarda los locators y expone las acciones del usuario; los tests solo llaman a esos métodos y verifican el resultado. Si cambia la UI, se corrige un solo archivo.



💡 Ángulo agéntico: el POM es un **guardrail**. Sin patrón el agente inventa selectores (drift); con patrón, conforma a una interfaz ya nombrada.

Screenplay Pattern y Patrones de API

Marcano & Palmer — más allá de POM para flujos complejos

Qué es: un patrón que escribe el test como una obra: **actores** con habilidades, como usar el navegador o llamar una API, ejecutan **tareas** y hacen **preguntas** sobre el sistema. El test se lee como la historia del usuario y las tareas se reutilizan entre actores.

- ▶ Ventaja sobre POM: flujos multi-actor y multi-rol son más naturales
- ▶ Ejemplo: ``Carlos.attemptsTo(EnviarMensaje.con("Hola"), VerificarRespuesta.noVacia())``
- ▶ **Patrones para API Testing:**
 - ▶ **Request Builders:** encapsulan construcción de requests
 - ▶ **Response Validators:** validan estructura de respuesta
 - ▶ **Contract Testing:** verifican que el contrato del endpoint no cambia



Ver ``fixtures/request-builders.ts`` en el repo ``qa-api`` — el mismo patrón en acción

05 — SECCIÓN

Pruebas de Agentes Inteligentes

El patrón Persona + Juez

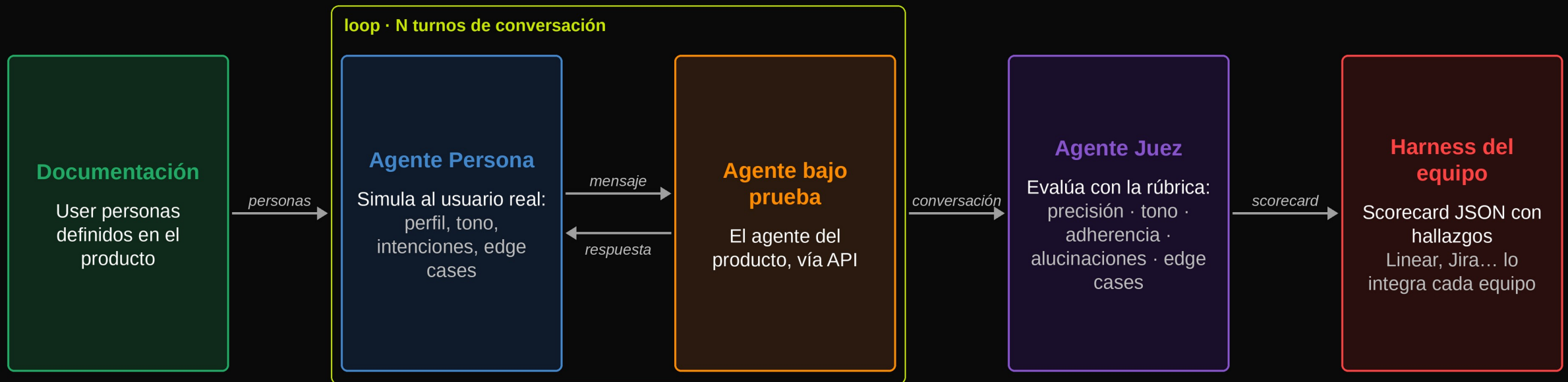
El desafío: respuestas no determinísticas

¿Cómo verificar algo que no siempre responde igual?

- ▶ Los agentes de IA producen **respuestas distintas** cada vez
- ▶ No se puede comparar con ``expect(respuesta).toBe("texto exacto")``
- ▶ Se necesita evaluación **semántica**: ¿es relevante? ¿es correcta? ¿es segura?
- ▶ Un test binario (pass/fail) no captura la calidad real del agente
- ▶ **Dimensiones que importan:**
 - ▷ Precisión factual · Relevancia · Tono · Adherencia al system prompt
 - ▷ Manejo de edge cases · Ausencia de alucinaciones

Patrón Persona + Juez

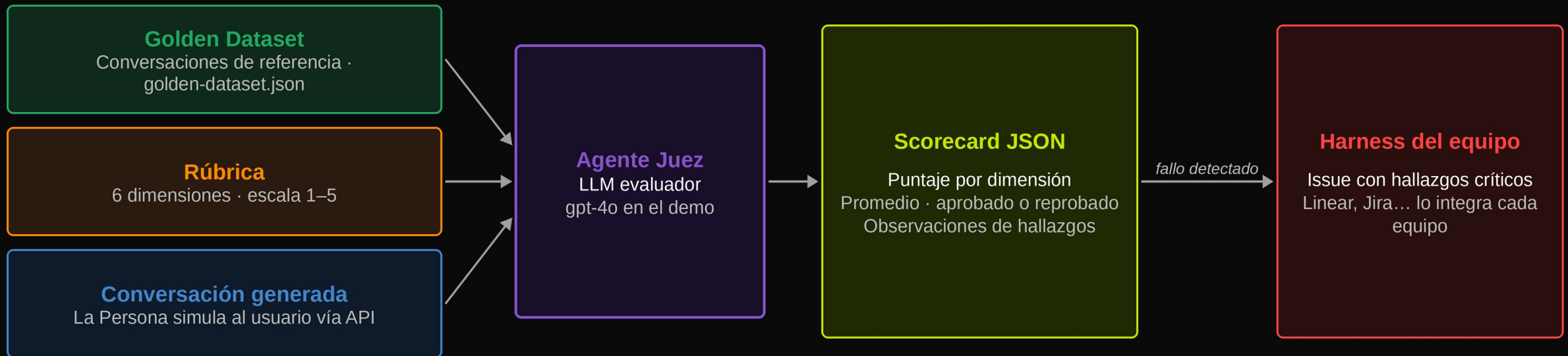
Evaluar agentes con agentes



💡 LLM-as-a-Judge lo formalizaron Zheng et al. (MT-Bench, NeurIPS 2023): GPT-4 concuerda con humanos ~80% — igual que dos humanos entre sí.

Golden Datasets

Medir al medidor — calibrar el juez antes de confiar en él



💡 Regla del campo: *nunca confíes en un juez que no hayas medido contra un gold set etiquetado*. El golden dataset es esa calibración.

DEMO

Ejecutar ``npm run eval`` en ``qa-agent-eval`` → ver conversación generada → ver scorecard con puntuaciones → aclarar: llevar hallazgos al harness (Linear, Jira...) lo integra cada equipo

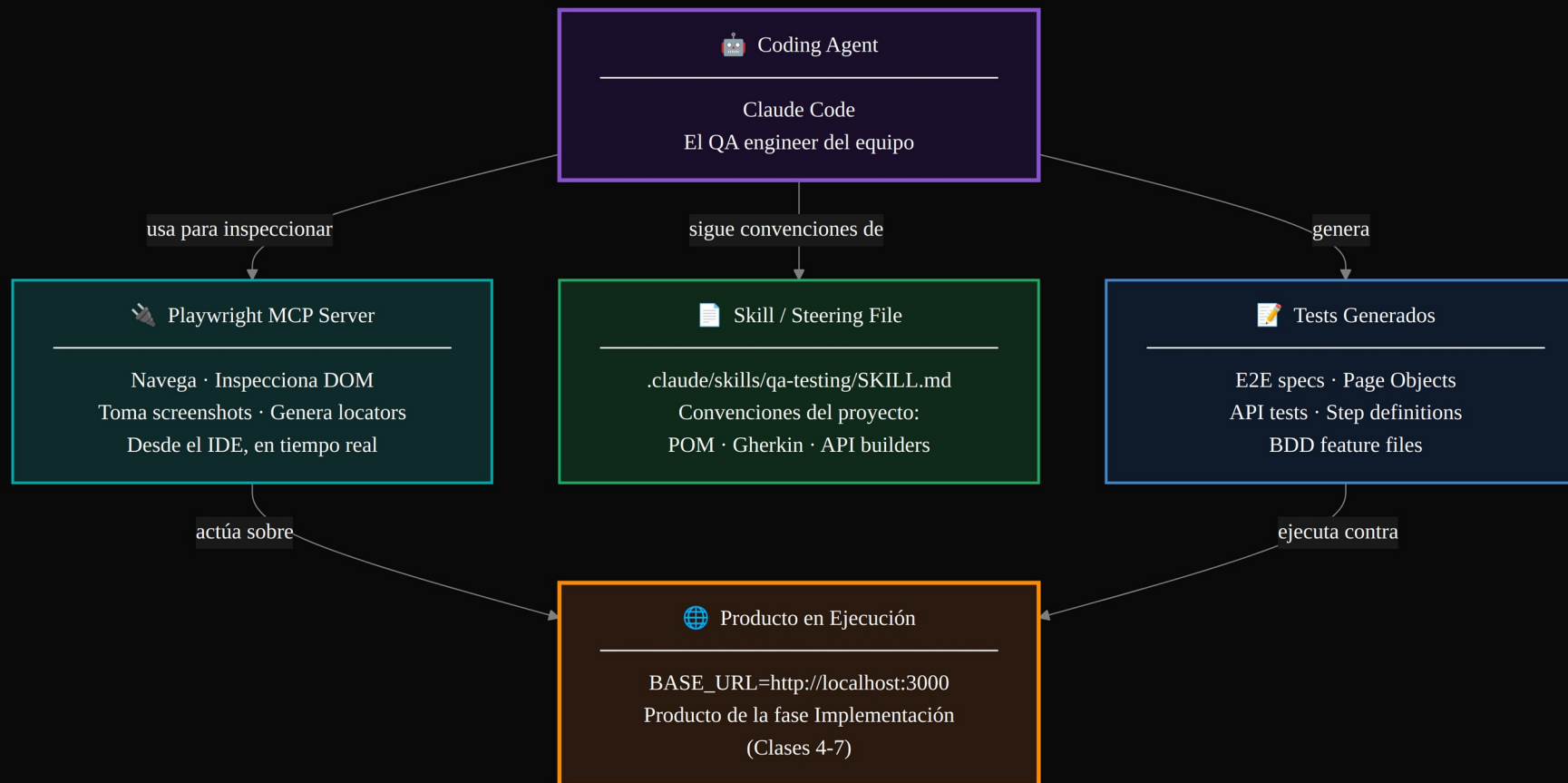
06 — SECCIÓN

El Stack de IA para Testing

Show & Tell en repositorio real

El Stack: Coding Agent + MCP + Steering File

Las tres piezas que lo hacen posible



El Steering File para QA

Las convenciones del proyecto en manos del agente

- ▶ Qué incluye ``.claude/skills/qa-testing/SKILL.md``:
- ▶ Estructura de carpetas (``tests/e2e/``, ``pages/``, ``steps/``, ``tests/features/``)
- ▶ Convenciones de naming (POM, specs, step definitions, features)
- ▶ Patrones obligatorios: POM primero, luego tests que lo consumen
- ▶ Configuración Playwright: ``trace``, ``screenshot``, ``baseURL``
- ▶ Instrucciones específicas al agente: usar MCP antes de generar locators



Es el documento de convenciones que los equipos siempre tuvieron y nadie leía — ahora el agente lo lee en **cada** prompt. La tesis en una pieza.



DEMO

Recorrer repo ``qa-e2e`` en IDE — ``playwright.config.ts`` · ``pages/`` · ``tests/e2e/`` · ``.mcp.json`` · ``.claude/skills/qa-testing/SKILL.md`` — la narrativa del proyecto

TRANSICIÓN

Kahoot

Validar comprensión antes del hands-on

- ▶ Pirámide de pruebas: niveles y trade-offs
- ▶ Ciclo TDD · qué es BDD
- ▶ Gherkin · POM · Screenplay
- ▶ Patrón Persona + Juez · Golden datasets

TAREA

Para la próxima sesión

Cobertura funcional completa de tu producto

- ▶ ☐ **Skill de testing** — ``.claude/skills/qa-testing/SKILL.md`` con convenciones de POM, Gherkin y API builders
- ▶ ☐ **E2E con POM** — al menos **2 flujos críticos** del producto cubiertos con Page Objects + tests Playwright
- ▶ ☐ **Tests de API** — al menos **1 endpoint clave** con request builders + contract tests
- ▶ ☐ **Evaluación de agentes** (si aplica) — golden dataset + rubric + ejecutar Persona + Juez al menos 1 vez



El objetivo es construir el **stack de QA personal** que vas a reusar en todos tus proyectos. Los tests son la red de seguridad antes de producción.